NeurIPS 2026 Education Track

Test-Time Compute Allocation

Search vs. Verify

Under a weak verifier and a mid-size budget, search wins, and the math tells you why.

预留作者信息

MOTIVATION

One Budget, Two Ways to Spend It

Test-time compute allocation is **the problem of dividing a fixed inference budget (a fixed number of model calls) between search and verification.**
Think about it: You have an open-source model (e.g. Qwen3-4B, ~4 GB quantized), MATH-500 competition problems, and a hard cap of **50 inference calls per problem**. How do you spend them?

Plan A: Search

Generate 50 unique answers, conduct a vote, and select the answer with the highest frequency as the final answer.

Plan B: Verify

Generate 5 distinct answers, have the model check each of these answers 9 times and score them, then select the answer with the highest average score as the final answer.

What does your intuition tell you? Many people pick Plan B: a model checking itself sounds smarter than a crowd of blind guesses. However, **some 2025 researches on weak verifiers say the opposite.**

This course will guide you through **verifying the above claims on your own**, and **evaluating how to choose between the two strategies** from a mathematical perspective.

MOTIVATION

Why the Answer Arrived in 2025

before 2025	ICLR 2025	COLM 2025	ICML 2025	NeurIPS 2025
Stuck on a problem? Have the model "think a little deeper" (chain-of-thought). But there was no systematic answer to how to split a fixed call budget.	Trained a "step-checker" (PRM) that grades each reasoning step. Finding: the best strategy depends on difficulty and budget.	Even a verifier with zero training helps, but only a little. First clue: verifier quality caps the gains.	With a huge budget, sampling + self-verification beat a specialist reasoning model (such as GPT o1-Preview).	Pairwise comparisons stay guaranteed even below accuracy rate $p = \frac{1}{2}$; one long chain beats many short ones exponentially.

In this course, our setup is: **a weak verifier (LLM self-scoring) + a moderate budget (number of inferences B = 50).** Under this setup, the trade-off is determined by a probabilistic model, and **the experiments in the workbook will allow you to verify this model for yourself.**

Tips: test-time compute allocation became a research question only recently, and **the answer depends on who judges the answers.**

CONCEPTS

What Counts as One Inference Call

One inference call = one LLM call producing one text (a candidate answer, or one verification judgment).

Voting costs nothing: counting how often each answer appears is free arithmetic.

Verification costs real calls: each of the N candidates is re-scored M times.

The budget identity

Search: $N \times (1 + M) = B$ **with** $M = 0$

That is, $N = B$

All the budget is used to generate candidate answers; every call grows the candidate pool.

Verify: $N \times (1 + M) = B$ **with** $M > 0$

Budget is split between generating and re-checking; different allocations give different results.

If the correct answer was never generated (for any reason), no amount of re-checking can find it.

CONCEPTS

Search or Verify? A Head-to-Head Comparison

Search—Majority Vote

Generate N candidates, pick the most frequent answer

Candidate pool (N independent calls)

A C A A ...

Majority vote
count answers, pick the most frequent

Final answer: A
most frequent candidate wins

$e^{(-N \cdot D)}$—exponential decay guarantee
when $p > \frac{1}{2}$, the vote-failure rate decays as $e^{(-N \cdot D)}$
(Chernoff bound, $D = KL(\frac{1}{2} \| p)$)

Voting consumes no inference calls
the N calls all go into generating candidates

Budget B

per problem

N generate · N·M verify

$$N \times (1 + M) = B$$

Same budget. Which wins?

measured: search wins under a weak verifier (B=50, LLM self-scoring)

Verification—Score Selection

Generate N candidates, score each M times, pick the best

Candidate pool (N independent calls)

A C B D ...

Verifier—score each candidate M times
LLM self-scoring 1-10, M independent re-scores

Final answer: C
highest average score wins

α / β—verifier error rates
α = P(wrong scored as correct) ≈ 0.62 (measured)
β = P(correct scored as wrong) ≈ 0.37—barely distinguishes

Gain limited by verifier quality
more re-scores (M) resample the same weak signal

MODEL

Modeling a Single Candidate

Crude bound: $P(all\ N\ wrong) = (1-p)^N \to 0$—but this only bounds the all-wrong case.

The real question is: how fast does the slim-majority failure probability decay?

Each independent candidate is correct with probability p (e.g. $p = 0.3 \to 1$ in 3 answers is right).

Independence is an assumption—candidates are separate calls, so errors do not have to align (until they do).

A majority vote fails when **more than half the candidates are wrong** (ties count as failure in this course's convention).

This is far weaker than "all N wrong": the $(1-p)^N$ bound covers only that extreme—failure needs just a slim majority of errors.

Why N first? Verification inspects the pool that was generated. With too few candidates, the right answer may never be in it.

MODEL

The Chernoff Bound: When Voting Is Guaranteed

If the single-candidate accuracy $\boldsymbol{p} > \frac{1}{2}$, the vote-failure probability decays exponentially:

$$\boldsymbol{P(vote\ fails) \leq e^{-N \cdot D}}, D = KL(\frac{1}{2} \parallel p)$$

The condition $\boldsymbol{p} > \frac{1}{2}$ is real:

$p = 0.5 \rightarrow D = 0 \rightarrow$ **no exponential guarantee exists**; the Chernoff premise fails.

This is a mathematical guarantee—no training, no labeling.

Take $\boldsymbol{p = 0.6}$, $\boldsymbol{N = 50}$ as example.

The formula promises: $P(vote\ fails) \leq 0.37$. Count exactly instead—ties count as failure—and the true rate is 0.098, **about 4 × lower.**

A bound is a safe promise, not a forecast: it shows the exponential trend, not the exact value.

When $\boldsymbol{p} \leq \frac{1}{2}$, voting has no p-only guarantee: its real effect depends on error spread. Experiment 3 will test this conclusion.

MODEL

When $p \leq \frac{1}{2}$: Voting Has No Guarantee, Verification Has a Ceiling

$p \leq \frac{1}{2}$: two mechanisms decide

① **Binomial fact (no spread needed):** counting ties as failure, $p < \frac{1}{2}$ makes voting worse than one sample: $p = 0.3, N = 3 \rightarrow$ 21.6% vs. 30%.

② **Error spread:** concentrated errors → the vote picks a shared wrong answer; spread errors → ties, decided by the tie rule.

Measured: Qwen3-0.6B (p = 0.46) majority vote at N = 10 reaches 0.57, **above its own single-candidate 0.46.**

Verification: the verifier has errors too

α = false-positive rate (wrong answer called right); β = false-negative (right answer called wrong).

Asking the same biased verifier M times does not improve α or β: **the ceiling is set by verifier quality, not by M.**

LLM self-scoring is a weak verifier: measured on Qwen3-0.6B self-scoring: $\alpha \approx 0.62, \beta \approx 0.37$.

Design note: re-asking the same judge M times averages out its noise; it cannot fix its bias. Improving α, β needs a better judge, not more passes.

Model gives conditions; measurements give answers. Both regimes ($p > \frac{1}{2}$ and $p \leq \frac{1}{2}$) show up in the real data in the further explanation.

EVIDENCE

Worked Example: The Headline Table

Strategy	Allocation (N, M)	Accuracy	95% CI	vs search
Search (majority vote)	50 candidates, 0 checks	0.853	95% CI [0.81, 0.89]	▲ best
Search + light verify	25 candidates, 1 check	0.717	95% CI [0.67, 0.77]	−13.6 pp
Verify (score selection)	5 candidates, 9 checks	0.753	95% CI [0.70, 0.80]	−10.0 pp

Qwen3-4B · 100-problem MATH-500 subset · budget B = 50 calls per problem

- Each accuracy is the mean of 3 independent runs (seeds): 0.853 = 256 correct answers out of 300.
- The brackets are 95% confidence intervals (95% CI): the plausible range of the true accuracy.
- All three rows spend the same budget, only the split changes: 50 candidates, or 5 candidates × 9 checks.
- You will reproduce this table with your own model in Experiment 1.

1. **Search beats pure verification by +10.0 pp**, not because judging is bad, but because 5 candidates rarely contain the right answer.
2. Search + light verify (0.717) lands near pure verification: one check on 25 candidates does not make up for losing 25 candidates, and 9 checks beat 1.
3. **The conclusion is limited to this configuration**: weak self-verifier, B = 50, and the same MATH-500 subset. Results can shift with the configuration.

EVIDENCE

Search Wins Across Three Models

Model	p	regime	Search (N = 50)	Verify (N = 5, M = 9)	Δ
Qwen3-4B	0.73	guaranteed	0.853	0.753	+10.0 pp
Gemma-3-4B	0.61	guaranteed	0.740	0.650	+9.0 pp
Gemma-3-1B	0.37	no guarantee	0.473	0.400	+7.3 pp

3 seeds × 100 problems; search wins on all 9 model–seed pairs (paired McNemar 9/9); 95% CIs exclude 0 everywhere; Δ from unrounded values.

Guaranteed regime ($p > \frac{1}{2}$): Qwen3-4B wins as the Chernoff bound predicts.

No-guarantee regime ($p \leq \frac{1}{2}$): Gemma-3-1B and Gemma-3-4B still win—spread errors make voting work in practice.

Tie rates at N = 50: 2.3% / 1.7% / 3.7% (Qwen3-4B / Gemma-3-4B / Gemma-3-1B). The lowest-p model ties most (3.7%); ties are decided by the vote rule, so voting keeps helping.

This is the p-spectrum lesson you will reproduce in the notebook experiments.

EVIDENCE

Reading the Table: Why Search Wins

The mechanism in brief

- Search raises N, so the right answer is more likely to be generated in the first place.
- Verification inspects a fixed pool. If the answer is not there, scoring it M times is wasted effort.
- **B = 50:** generating 50 candidates beats generating 5 and judging them.
- **Score is secondary:** what matters is whether the right answer was generated at all.

The verifier barely separates right from wrong

Judge accuracy gap 63.8% vs 62.2% *(0.6B self · zero discrimination)*

Selection gain 0.753 vs baseline 0.766 *(Qwen3-4B · no gain)*

False positive rate $\alpha \approx 0.62$ *(0.6B self · wrong called right)*

False negative rate $\beta \approx 0.37$ *(0.6B self · right called wrong)*

Mean scores 7.34 vs 6.97 *(Qwen3-4B · scores barely separate)*

With a weak verifier, "polish harder" fails: $\alpha \approx 0.62$ means most wrong answers are still called correct after M re-checks.

COMPARE

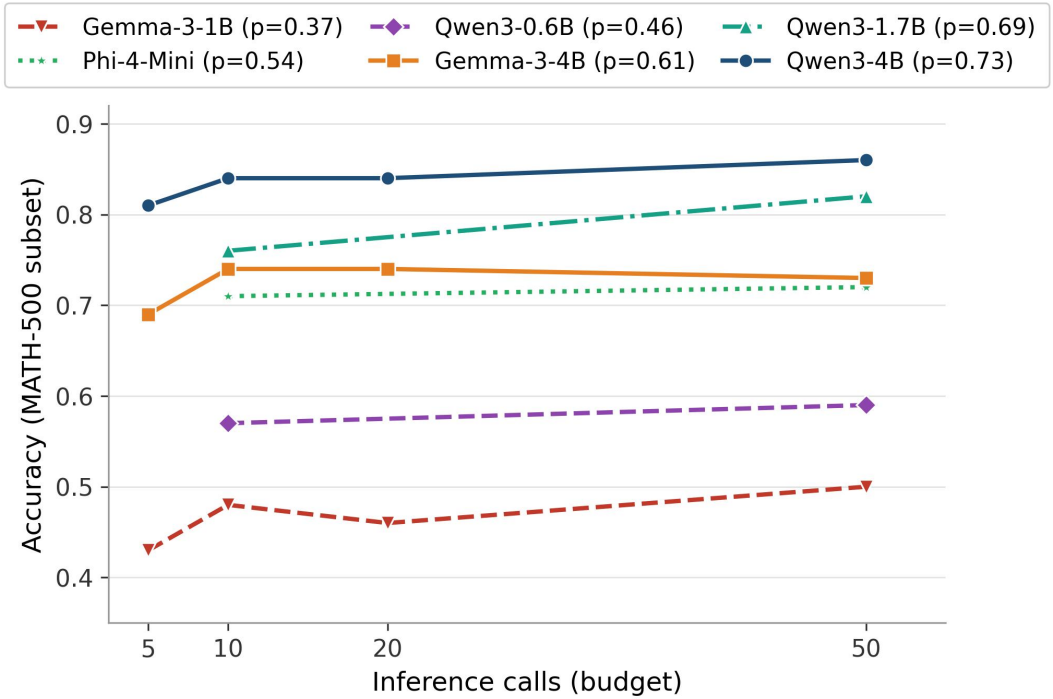
Two Strategies at a Glance

Dimension	Search (majority vote)	Verify (score selection)
What it does	generate many answers, pick the most frequent	judge each answer, pick the highest score
Why accuracy rises	more votes ⟶ more likely correct; all-wrong shrinks	the judge filters obvious mistakes
Main risk	all candidates share the same error → vote is useless	the judge is wrong ⟶ filtering is random
Failure probability	$\approx e^{-N \cdot D}$ when $p > \frac{1}{2}$	set by verifier quality (α, β)
Best when	mid budget (10–100 calls), diverse candidates	huge budget (100+) and a strong judge
Needs	nothing but statistics	a trained verifier

Search is strength in numbers; verification is hiring an expert. Hire the expert only when they are actually good.

COMPARE

The p-Spectrum: Six Models, One Rule



Main models (Gemma-3-1B / Gemma-3-4B / Qwen3-4B): full 23-config grid, seed 0; the other three (Phi-4-Mini / Qwen3-0.6B / Qwen3-1.7B): 6-config student grid (search N=10/50 + verify splits). Headline numbers on p. 9 are 3-seed means.

- **Six models, p from 0.37 to 0.73**
- **Higher p → higher curve in the guaranteed regime ($p > \frac{1}{2}$)**: voting scales where the math promises.
- **Below $\frac{1}{2}$ shapes vary: error spread decides;** voting still beats single candidates everywhere.
- *The guaranteed regime vs. the no-guarantee regime is the course's core lesson.*
- **p is your model's single-candidate accuracy: be sure to measure it first, don't guess it.**

Where do you stand? Find your own model's p in Experiment 1, then predict where its curve should sit.

COMPARE

Verifier Quality Sets the Ceiling

Verifier strength	Representative	Evidence	Lesson
Weak / none	LLM self-scoring, majority vote	this course's setting	search more reliable
Medium	zero-shot generative verifier (GenRM-Base)	COLM 2025	limited value
Strong	fine-tuned PRM / GenRM-FT	ICLR 2025 + COLM 2025	verification shines
Huge budget + deep self-check	sampling + self-verification (Verification@200)	ICML 2025	self-verify beats voting

Verifier quality decides the value of verification. This course's claim sits at the weakest end of the spectrum (top row of this table), and the notebook experiments test exactly this.

ICLR 2025 used a strong trained PRM and found verification-guided search winning at mid budgets. The two results complement each other.

Design implication: with only a weak self-verifier, every call spent on M is borrowed from N—it shrinks the candidate pool while the judge's α/β cap the return. Raise N first; invest in M only when the verifier is strong.

DECIDE

Decision Tree (1): Know Your p

Given a task, budget B and model: first estimate the single-candidate accuracy p (5~10 problem probe).

Measure p first

$p \leq 0.5$	$0.5 < p \leq 0.6$	$p > 0.6$
no mathematical guarantee	**gray zone: guarantee weakens**	**Chernoff guarantee active**
• Probe: 5~10 problems, majority vote vs single sample • Voting can still win; e.g. 0.6B ($p = 0.41$) reaches 0.61 at N=10 • If voting loses: single samples, or borrow a stronger verifier	• Measured in this band: Phi-4-Mini (p ≈ 0.54) • Probe 5~10 problems and decide from measured behavior • Whichever side wins, treat it as your p going forward	• Voting scales with N: the guarantee is real • Measured: 1.7B 0.82 and 4B 0.86 at N = 50 • Then follow the budget branch (next page)

The threshold (1/2, 0.6) is a modeling convention, not a hard law. Measure first, then decide. Remember that the tree is a hypothesis to test, not a verdict.

DECIDE

Decision Tree (2): Budget × Verifier

Budget B	Recommendation	Why
$B < 10$	pure search (N = B)	too few candidates to share with a verifier
$10 \leq B < 100$	verifier weak or unknown → pure search; verifier strong → search-first (e.g. N = 25, M = 1)	guaranteed voting beats a weak judge
$B \geq 100$	strong verifier → split (e.g. N = 50, M = 1); no good verifier → pure search	marginal value of extra candidates drops

Reading the branches: the pattern is "generate first, judge later". $B < 10$ leaves no room to split; $B \geq 100$ with a strong judge makes splitting worthwhile. Without a good judge, search always wins.

Caveat: The 10/100 thresholds are based on experimental experience, and your probe may shift them. That is exactly what Experiment 3 tests.

HANDS-ON

Experiment 1: Budget Efficiency

What you do

Fix B = 50 on the shared 6-problem MATH-500 subset, and run the six representative configs: search (N = 10, N = 50) and verify (N = 5 & M = 1, N = 5 & M = 9, N = 10 & M = 4, N = 25 & M = 1).

Plot your model's budget–accuracy curve (≥ 2 points).

Optional sub-experiment: score the same candidates with your model (self) vs Qwen3-4B (unified)—verifier quality isolated.

Watch the demo, then answer

- **Where do the two curves cross?** How much budget does verification need to catch up with N = 50 voting?
- Does your p (measured from single samples) predict the shape?

Entry: notebooks/01_budget_efficiency.ipynb. Pick one model that fits your device; cached six-model curves are provided for the rest.

Rubric check (goal 2): *"plot ≥ 2 points on your own curve and interpret the six-model family, mentioning p ordering."*

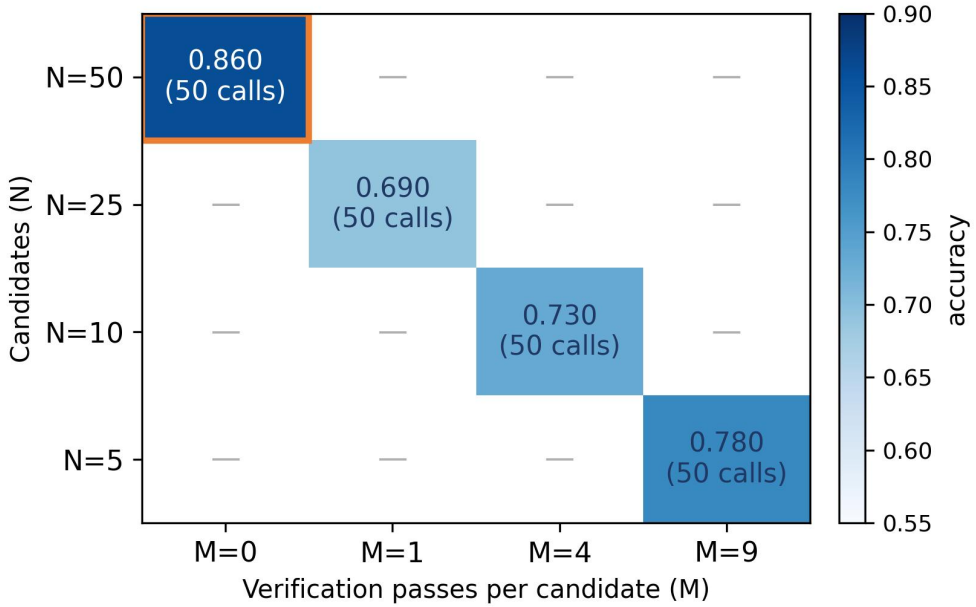
Expected direction: With this configuration, the search should succeed. If your results differ, **trust your data, analyze it, and explain the difference.**

HANDS-ON

Experiments 2–3: Optimal Ratio & New Tasks

Exp. 2: fix B = 50, try the four legal (N, M) cells—{(5,9), (10,4), (25,1), (50,0)}. Where is the best cell?

You run these four on the shared 6-problem subset; the map shown is the cached 100-problem grid (qwen3-4b). $N \times (1 + M) = 50$ keeps the legal cells on the diagonal.



Orange box: the best cell, N=50 pure search (0.86)

Exp. 3: predict on GSM8K, then check

- Based on the model, predict the optimal ratio on a new task (GSM8K).
- Estimate p from the included 5~10 problems probe. But p in GSM8K can differ from MATH (it shifts by model: lower for Gemma-3-4B, higher for Qwen3-4B), so measure it, don't assume it.
- **Wrong prediction?** Find which assumption broke (independence? verifier quality? budget scaling?), and that is the lesson.

Rubric check: *"solve for N via the bound" and "propose a (N, M) split stating p, budget, verifier quality."*

Cross-validate with the materials: your heat map & the cached grid; your GSM8K prediction & the decision tree. Trust your raw experimental data.

HANDS-ON

Experiment 4: Audit the Baselines

1. budget coverage	Does the paper sweep small → large budgets, or cherry-pick one? (ICLR 2025 sweeps; check ours: B = 50 only)
2. faithful baselines	Is Majority voting / best-of-N weighted implemented as defined: sampling counts, scoring, difficulty buckets?
3. verifier disclosure	Is the PRM training setup stated? (ours: self-scoring 1-10, threshold ≥ 7— disclosed)
4. one flaw to point out	At least one baseline defect must be identified, e.g. Self-Consistency naming or budget gaps.

Apply the same lens to this course: judge parse-failure rate 5.2% (final disclosed); headline accuracy stable across NaN-handling choices. Tie rules and CI conventions on p. 9–10.

Full audit checklist and this course's own disclosures are in notebooks/04_baseline_review.ipynb.

WRAP-UP

Takeaways & Your Rubric

- **Search wins by maximizing the candidate pool:** the right answer must first be generated, and verification cannot find what was never generated.

- **Verifier quality sets the ceiling:** weak self-scoring cannot fix a bad pool; strong judges change the calculus (see the spectrum).

- **Model first, measure second:** $p > \frac{1}{2}$ gives a guarantee; $p \leq \frac{1}{2}$ needs a probe. The decision tree turns both into a plan.

Goal	Rubric
1. calculate	Given p and a target failure rate, solve for N via the Chernoff bound (must show the $D = KL$ step).
2. plot & read	Plot ≥ 2 points on your own budget–accuracy curve and interpret the six-model family (must mention p ordering).
3. propose	Propose a (N, M) split for a new task with a model-based reason (state p, budget, verifier quality).
4. audit	Name at least one baseline flaw in the audited paper (e.g., SC replication or budget coverage).
5. decide (L2)	Walk the decision tree for a new task and state the p estimate that drives the choice.

Further reading

ICLR 2025: Scaling LLM Test-Time Compute Optimally

COLM 2025: When To Solve, When To Verify

ICML 2025: Sample, Scrutinize and Scale

NeurIPS 2025: Let Me Think!

NeurIPS 2025: Provable Scaling Laws

Links: course GitHub README